

实验与机试指南

© 2023-2025 山东大学 软件学院 实验教学管理中心

Talk is cheap. Show me the code.

Linus Torvalds (2000-08-25)

评测环境

编程语言：C, C++, Java, Python, JS, SQL, Bash. 实验或题目可能限定语言。

编译器版本：

- gcc:15.1.0, C(23), C++(23)
- openjdk:25
- pypy:3.11
- node:23.11.0
- postgres:17.4
- ubuntu:25.04

编译选项：

- C/C++通用选项 (CFLAGS)
 - -O2 二级优化。 -g 生成调试信息。
 - -DONLINE_JUDGE 定义 ONLINE_JUDGE 宏，用于屏蔽调试语句。
 - -pedantic-errors 符合ISO C/C++标准，禁用GNU扩展（例如变长数组）。
 - -Wall 警告所有常见缺陷。 -Wextra 警告更多缺陷。
 - -Wshadow 警告变量遮蔽（局部变量覆盖外层作用域同名变量）
 - -Wconversion 警告隐式类型转换（如 int 到 short，long 到 double）
 - -Wduplicated-cond 重复的条件。
 - -Werror **警告视为错误，必须修正。**
- gcc -x c -std=c23 \${CFLAGS} -o \${elf} \${src} -lm
- g++ -x c++ -std=c++23 \${CFLAGS} -Wefc++ -o \${elf} \${src}
- javac --release 25 -encoding UTF-8 -sourcepath . -d . \${src}

常见问题

- 每题最多提交32次。“编译错误”或“样例未过”可以点“提交时间”查看编译和评测信息，修改代码后再次提交，不重复计数。
- 实验和考试模式无罚时，多次提交取最高得分。
- 源码限长64KB，编译时限30s，内存限额2GB，程序不超过30MB。超限将判编译错误CE。
- 题目如无特殊说明：
 - 所有语言的时空资源限制相同。
 - 源码编码为UTF-8。
 - 仅使用ASCII范围的字符。
 - C/C++的 `main()` 返回类型必须是 `int`，且必须返回0。
 - 程序结束必须返回0，也不可抛出异常，否则将判运行期错误RE。
 - 程序应从 `stdin` 读入数据，向 `stdout` 输出结果。评测系统忽略 `stderr` 的内容，且不判错。
- 不要输出多余信息，比如 `cout << "please input:"`；，否则将判答案错误WA。
- 输入格式无需检查。例如“每行两个整数，间隔一个空格”，只需直接读入两个整数 `cin >> a >> b`；，不会缺少数据项，不用检查非整数的情况，也不用读入字符串再转整数。

审题

- 通过样例只是说明对题意有基本的了解，不代表程序正确。测试用例覆盖的情况更多、更复杂。
- 仔细阅读题目规格，分析隐含意图：
 - **整数**：考虑负数、0的情况。
 - **非负整数**：考虑0的情况。
 - **正整数**
 - `int32`：通常有 $[-2^{31}, 2^{31} - 1]$ 边界数据。注意加法和乘法溢出，考虑用 `long`。
 - `int64`：通常有 $[-2^{63}, 2^{63} - 1]$ 边界数据。
 - “数值不超过50000”：乘法可能溢出。
 - “数■不超过100000”：不能用平方阶或更慢的算法。
 - 数组下标从0开始。题面描述数据项、行、列时可能从1开始计数，需注意区分。
- 程序设计实验题目性能要求宽松，默认1秒时限、1GB内存足够使用。若时间超限TE或内存超限ME需要检查程序是否过于低效。

I/O

- 数据项十万以上如用C++IO流需考虑优化，例如 `ios::sync_with_stdio(false)`；，`cin.tie(0)`；，`cout.tie(0)`；，代价是不能与C语言的IO库函数混用。
- 如果题目要求整行读入，那么数据行的开头、中间、结尾可能有空白符（不限于空格），必须整行而不能逐段读取。

- C++:

- 输出定点小数: `#include <iomanip> , cout << fixed << setprecision(6) << x << endl;`
- 整行读入: `for(string s; getline(cin, s);) {}` 注意不是 `cin.getline()`
- 从按项读入转为整行读入时, 前项数据会残留换行, 导致多读一个空行。处理方法:

```
int n;
cin >> n; // 读完数值、停在换行处。
{ string t; getline(cin, t); } // 把读入n之后残留换行或空白字符耗掉
for(string s; getline(cin, s); ) {}
```

- `cin.ignore()` 略过下个字符 (任意字符, 不限空白), 也可略过多个字符:
- `cin.ignore(100, '\n');` 略过最多100字符, 或到换行为止 (略过换行)。
- `cin >> ws` 忽略后续空白符直到非空字符位置。如果下行数据开头有空白符, 不能用这种写法。

- Java:

- 输出定点小数: `System.out.printf("%.6f\n", 0.123456789);`

数据类型

- 评测环境为64位Linux系统, C/C++的地址、指针、`long` 类型都是64位、8字节。
- 浮点数运算通常可以放心使用 `double`, 无需考虑与 `float` 的性能差异。
- 程序的数据范围有限, 与理想中的数学不同。
 - 比如二分范围 `1 <= h`, 下列写法不等价:
 - `m = 1 + (h - 1) / 2;` 可靠, 对下标和指针都适用。
 - `m = (1 + h) / 2;` 可能溢出, 且不适用于指针
 - 比如素数筛:
 - `for(int j = i * i; j < n; 或 for(int j = i + i; j < n;` 可能溢出。
 - `for(int j = i, r = (int)sqrt(n); j <= r;` 较可靠。
- 如果本机运行正常, 评测报运行期错误 (RE), 可能有**变量未初始化**, 或数据范围过小。在Windows环境用VC++编程易出现该类疏漏。
- 数组不能过大。全局数组不宜超过千万, 局部数组不宜超过十万。反例: `int a[1e9];`
- 高精度运算如题目未作限制, 可以考虑用Java `BigInteger`, `BigDecimal`, 或用Python解题。
- 整型常量仅限`int32`。例如 `123456789012345` 超出`int`范围, 需要加 `L` 或 `UL` 后缀。
- 整型常量实际仅限正数。例如 `-1` 实际是常量 `1` 取负。
 - 如果直接写 `-2147483648` 则会报错, 因为 `2147483648` 也超出`int32`范围, 应写成 `-2147483647 - 1`。
 - 更好的办法是用 `<climits>` 等标准库定义的常量。
- C++的 `vector` 长度和下标都是无符号数, 与数组和指针不同。

代码格式

- 规范缩进，帮助理解程序结构。
- C/C++/Java的控制结构用大括号，即使单语句也不要简写。

```
for(;;) {  
}
```

- 判断分支总是对称设计，以防遗漏判断情况或条件组合：

```
if(...) {  
} else {  
    // 即使未用也注释理由。  
}
```

- 区间尽量设计为**左闭右开**，拼接时不留“空隙”。

```
for(int i = 0; i <= e; ++i) { // 反例：写出小于等于要打起精神，仔细检查，是否越界。  
}
```

- 优先用 `==`，不要求顺序（偏序关系），其次用 `<` 判断。其余4种关系算符可用前两种组合构成。
- 避免重复的函数调用。
 - 反例：`for(int i = 0; i < v.size(); ...`
 - 正例：`for(int i = 0, L = static_cast<int>(v.size()); i < L; ...`
 - 如果循环中改变了 `v` 的长度，正例的写法则有问题，但这种情况需要修正的是算法设计：不该使用计数循环。

评测流程

为帮助理解裁决结果，简单介绍评测的流程：

- 用户选择语言类型，提交解题代码。系统将代码保存到数据库，状态为**等待评测**。
- 评测机向服务器请求待测人物。服务器按提交时间安排评测，将状态改为**提交评测**。
- 评测机获取被测源码，验证题目版本、样例、测试用例可用性。若数据缺失、系统故障等：
 - **SE: System Error**（系统错误，非用户原因。无法评测：缺少输入、输出、验证器等）
- 系统在1核、3GB内存的容器环境编译代码，时限30秒。若源码语法错误、选错语言类型、编译耗时过长、消耗内存过多、无法生成目标文件等，报告**编译错误**：
 - **CE: Compile Error**（编译错误：语法错误等。编译超限：耗时太长或消耗太多内存）
- 评测机创建评测容器，挂载数据目录。
 - 容器环境为1核CPU、3GB内存。

- C/C++栈内存限额设置为内存限额，与静态数据、动态分配合计用量。
- JVM栈内存限额64MB，递归过深可能栈溢出。
- 若程序全局或静态数据过大导致无法启动，可能判为**运行期错误RE**而非内存超限（ME）。
- 每题有几个样例，最多100个正式用例（也称测试点，有对应分值）。每个用例为输入文件 `.in` 和答案文件 `.ans`。系统按照“样例、自测用例、正式用例”的顺序依次评测。

每个用例运行一次程序：

- 容器中的监控程序启动被测程序，将输入文件重定向到被测程序的 `stdin`，将程序输出（`stdout`）重定向并保存到文件。
- 程序正常运行，但试图调用受限的系统功能（文件、进程、网络等）：
 - **FE: Function Error**（功能受限：尝试被禁止的系统调用）
- 程序被信号终止、或抛出异常、或主程序退出状态码非0：
 - **RE: Runtime Error**（运行时错误：栈溢出、段错误、除0错、无效指令、异常终止等）
 - 鼠标指向 `SE` 标签可以查看具体错误，例如信号、返回值等信息，确定调查方向。
- 程序运行期间，系统定时测量**运行时长、CPU时长、内存用量峰值**，如超出题目限额则终止进程。
- 程序运行时长超出题目或容器限额而被终止：
 - **TE: Time Limit Exceeded**（时间超限：算法效率过低、无限循环、等待输入等）
- 程序内存用量超出题目或容器限额而被终止：
 - **ME: Memory Limit Exceeded**（内存超限：数组过大、分配对象过多等）
- 程序输出过多超出文件限额而被终止、或输出明显超过答案长度（2倍+1KB）：
 - **OE: Output Limit Exceeded**（输出超限：判错输入结束条件，无限循环不断输出）
- 程序正常结束，外部评测服务收到结果，与答案文件进行比对。
 - 输出结果与答案有实质性不同、或特判被拒：
 - **WA: Wrong Answer**（答案错误：逻辑错误、边界条件未处理、特殊情况未覆盖等）
 - 输出结果与答案不同，但忽略空白符差异后与答案一致：
 - **PE: Presentation Error**（格式错误：空格、制表、换行等空白符有差异）
 - 程序输出与答案相同，或特判通过：
 - **AC: Accepted**（解答正确，代码被接受）

TE, ME, OE 均属于资源受限，并无明确的触发顺序约定。

若某个样例结果错误，则略过后续正式用例，状态为**样例未过**。

若样例全部通过，则状态为**评测完成**，并按照通过的正式用例计分。

更新历史

- 2025-04-28

- 去掉了编译选项：`-Wsign-conversion` 警告有符号/无符号隐式转换。（如 `vector` 长度或下标用 `int`）

发现混用 `int/vector<>::size_type` 的情况很普遍。如确实误用负数也会爆出运行期错误，系统不背这个锅了。

- 2025-04-27

- C++启用标准库模块特性（`module`），支持 `import std;` 写法。
- 评测进度细化到测试点，代价是DB更新多一数量级。

- 2025-04-26

- 编译器升级：GCC-15.1(C23, C++23), OpenJDK-25, PyPy-3.11

- 2025-04-20

- 语言版本升级：C23, C++23, Java24

- 2025-03-28

- 开放C17, C++20, Java21, Python-3.12四种语言。

- 2025-03-16

- 限定C++20, Java21。